

FAQAI

AI & ML FAQ Chatbot

Project Documentation

CodeAlpha Internship - Task 1

Submitted By: Fatima Eman

Student ID: CA/DF1/192788

Full-Stack AI & ML Chatbot Web App

Table of Contents

1. Project Overview.....	3
2. Features.....	3
3. Technology Stack.....	4
4. Architecture.....	4
5. Conversation History (Session-Based, No Login Required).....	6
6. NLP Matching Engine (TF-IDF + Cosine Similarity)	6
7. API Documentation.....	7
8. Error Handling & Testing	7
9. Setup & Local Installation	8
10. Deployment.....	8
11. Known Limitations	9
12. Possible Future Improvements	9
13. User Interface.....	11

1. Project Overview

FAQAI is a full-stack AI & Machine Learning FAQ chatbot web application built to fulfil CodeAlpha Internship Task 2: a chatbot that collects FAQs on a chosen topic, preprocesses them using NLP techniques, and matches user questions to the most relevant answer using Cosine Similarity. The project extends the base requirement with a production-style architecture: a FastAPI backend running a real NLP pipeline (NLTK + Scikit-learn), a category-organized dataset of 43 FAQs, session-persisted conversation history, and a fully custom, responsive React interface.

The application is structured as two independently deployable services — a FastAPI backend exposing a REST API, and a React (Vite) frontend that consumes it — designed for separate hosting (Render + Vercel) behind a single public URL for the frontend.

1.1 Original Task Requirements

- Collect FAQs related to one topic or product (questions and their answers).
- Preprocess the text using NLP libraries like NLTK or SpaCy (tokenize, clean, etc.).
- Match user questions with the most similar FAQ using techniques like cosine similarity or intent matching.
- Display the best matching answer as a chatbot response.
- Optional: create a simple chat UI for user interaction.

1.2 What Was Built Beyond the Base Requirement

- A FastAPI backend with a modular services/routers architecture, rather than a single script.
- 43 hand-crafted FAQs organized into 9 categories (Artificial Intelligence, Machine Learning, Deep Learning, NLP, Computer Vision, Generative AI, LLMs, Python, Data Science), stored in JSON rather than hardcoded.
- A full React + Tailwind CSS single-page application — landing page, dark/light mode, and a complete chatbot interface — rather than a minimal UI.
- Category browsing sidebar, keyword search across all FAQs, and curated trending/popular questions.
- Session-persisted conversation history (no login required) with TXT/PDF export.
- Voice input (Speech-to-Text) and Text-to-Speech playback of answers.
- Bookmark/favorite FAQs, persisted locally across sessions.
- A confidence score surfaced on every matched response, and a smart fallback that never guesses when similarity is too low.
- Responsive design audited and refined across mobile, tablet, and desktop breakpoints.

2. Features

Area	Details
FAQ dataset	43 FAQs across 9 categories, stored in JSON (not hardcoded in source)
NLP preprocessing	Lowercasing, tokenization, punctuation removal, stopwords removal, lemmatization (NLTK)

Matching engine	TF-IDF vectorization + Cosine Similarity (Scikit-learn); confidence score returned with every response
Fallback handling	Similarity below threshold (0.25) triggers a friendly “couldn’t find a relevant answer” message — never guesses
Chat interface	User/bot bubbles, timestamps, auto-scroll, typing indicator, Enter-to-send
Categories	Sidebar navigation across 9 categories with live FAQ counts
Trending questions	Curated popular questions (Transformer, RAG, Overfitting, Gradient Descent)
Conversation history	Session-persisted (sessionStorage), Clear Chat button, no login required
Voice & audio	Speech-to-Text input, Text-to-Speech playback of bot answers
Utilities	Copy-to-clipboard, export chat as TXT/PDF, keyword search, bookmark/favorite questions
Theming	Dark/light mode toggle, custom glassmorphism design system
Responsiveness	Audited across mobile (375px), tablet (768px), and desktop (1280px+)

3. Technology Stack

3.1 Frontend

- React 19 + Vite — component architecture and fast dev/build tooling
- Tailwind CSS v4 — utility-first styling, custom design tokens for the brand palette
- Framer Motion — scroll and interaction animations
- Axios — REST API client
- React Router — client-side routing (landing page / chatbot page)
- jsPDF — client-side PDF generation for chat export
- lucide-react — icon set

3.2 Backend

- Python 3 + FastAPI — REST API framework
- NLTK — tokenization, stopword removal, lemmatization
- Scikit-learn — TF-IDF Vectorizer and Cosine Similarity
- Pydantic — request/response validation
- python-dotenv — environment variable management
- Uvicorn — ASGI server

4. Architecture

The project is organized into two top-level folders — backend and frontend — deployed as two independent services that communicate over a REST API.

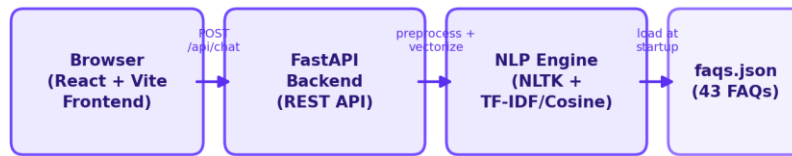


Figure 1 — Request flow from browser to NLP matching engine

Request flow for a chat message:

- 1. User types a question in the chat interface (frontend/src/components/ChatInput.jsx).
- 2. Frontend sends a POST request to /api/chat with the message text.
- 3. FastAPI backend runs the NLTK preprocessing pipeline on the message and vectorizes it using the already-fitted TF-IDF vectorizer.
- 4. Cosine similarity is computed against every FAQ vector; the highest-scoring FAQ is selected if it clears the confidence threshold.
- 5. The matched answer, category, and confidence score are returned as JSON and rendered as a chat bubble — or, if nothing clears the threshold, a fallback message is returned instead.

5. Conversation History (Session-Based, No Login Required)

A key design goal was that each visitor’s conversation should persist across a page refresh and feel personal, without requiring user accounts or a server-side database of personal data. This was solved entirely on the client:

- Every message exchanged in the chatbot is stored in the browser’s sessionStorage under the key faqai-chat-history.
- On page load, the chat window automatically restores the previous session’s messages — refreshing the page does not lose the conversation.
- Closing the browser tab clears sessionStorage, so no conversation data persists beyond that session, and nothing is stored server-side.
- The Clear Chat button resets both the in-memory state and sessionStorage immediately.

Bookmarked FAQs use a separate, longer-lived store (localStorage, key faqai-bookmarks) so favorites survive across sessions, while the conversation itself remains session-scoped by design.

Trade-off: because history lives in sessionStorage rather than a database, it is scoped to a single browser tab and is not shared across devices. This matches the assignment’s “no login required” expectation while keeping the backend fully stateless with respect to user data.

6. NLP Matching Engine (TF-IDF + Cosine Similarity)

This is the core of the project — real statistical NLP, not keyword matching or hardcoded rules. All 43 FAQ questions are preprocessed once at backend startup and vectorized into a TF-IDF matrix. Every incoming user message goes through the identical pipeline before comparison:

- Lowercasing — normalizes case so “Overfitting” and “overfitting” are treated identically.
- Tokenization — splits text into individual word tokens (NLTK word_tokenize).
- Punctuation removal — strips punctuation before comparison.
- Stopword removal — discards common low-information words (“what”, “is”, “a”, ...).
- Lemmatization — reduces words to their dictionary form (NLTK WordNetLemmatizer), e.g. “running” → “run”.

The processed query is vectorized with the same fitted TF-IDF vectorizer, and cosine similarity is computed against every FAQ vector. The highest-scoring FAQ is returned only if its similarity clears the configured threshold (default 0.25); otherwise the chatbot responds with a fallback message rather than guessing.

Known limitation of this approach: because matching is purely lexical (word-overlap based), an abbreviation like “NLP” will not match a FAQ phrased as “Natural Language Processing” unless the literal words overlap after preprocessing. This is documented further in Section 11.

7. API Documentation

Method	Path	Purpose
GET	/api/health	Health check, returns FAQ count
GET	/api/faqs	All FAQs (optional ?category= filter)
GET	/api/faqs/categories	Categories with FAQ counts
GET	/api/faqs/popular	Trending / popular FAQs
GET	/api/faqs/suggested	One suggested question per category
POST	/api/chat	Send a message, receive the matched answer + confidence score

7.1 Example Request — POST /api/chat

```
{ "message": "what is overfitting" }
```

7.2 Example Response

```
{
  "matched": true,
  "answer": "Overfitting occurs when a model learns the training data too well, including its noise and outliers, causing it to perform poorly on new, unseen data.",
  "question": "What is overfitting?",
  "category": "Machine Learning",
  "confidence": 1.0
}
```

8. Error Handling & Testing

The following cases were explicitly tested against the running backend and frontend:

- Empty or whitespace-only message → 422 validation error (“Message cannot be empty or whitespace only”)
- Message exceeding 500 characters → 422 validation error, rejected before matching
- Off-topic / unrelated query → returns `matched: false` with the fallback message instead of guessing (verified: confidence 0.0 for out-of-domain input)
- Backend unreachable from the frontend → chat UI shows a friendly “couldn’t reach the server” bot message instead of crashing
- CORS configuration → verified the allowed origin (`http://localhost:5173`) receives an `access-control-allow-origin` header; other origins are blocked
- FAQ data file missing or corrupted at startup → `FAQService` raises a clear `FileNotFoundError` immediately rather than starting with a broken matcher
- Category filter with a non-existent category name → returns an empty list rather than erroring
- Voice input on unsupported browsers → the microphone button is hidden via feature detection rather than throwing an error

9. Setup & Local Installation

9.1 Requirements

- Python 3.10 or later
- Node.js 18 or later
- npm

9.2 Steps

Backend (Terminal 1):

```
cd backend
python -m venv venv
venv\Scripts\activate    (Windows)
source venv/bin/activate (macOS / Linux)
pip install -r requirements.txt
copy .env.example .env   (Windows: copy | macOS/Linux: cp)
uvicorn app.main:app --reload
```

Frontend (Terminal 2):

```
cd frontend
npm install
copy .env.example .env
npm run dev
```

Backend runs at `http://127.0.0.1:8000` (interactive docs at `/docs`). Frontend runs at `http://localhost:5173`. Both servers must be running simultaneously for the chatbot to work.

10. Deployment

The frontend and backend are deployed as two separate services, connected via an environment variable rather than a hardcoded URL.

10.1 Recommended Hosts

Backend → Render.com:

- Push the project to a GitHub repository.
- Create a free account on Render.com and connect the repository.
- Root Directory: backend
- Build Command: `pip install -r requirements.txt`
- Start Command: `uvicorn app.main:app --host 0.0.0.0 --port $PORT`
- Instance Type: Free
- Environment Variable: `ALLOWED_ORIGINS` set to the deployed Vercel URL

Frontend → Vercel:

- Import the same GitHub repository into Vercel.
- Root Directory: frontend
- Framework Preset: Vite
- Environment Variable: `VITE_API_BASE_URL` set to the deployed Render URL

10.2 Free-Tier Considerations

- Render free instances sleep after ~15 minutes of inactivity; the first request after idle can take 30–60 seconds to respond.
- Vercel’s free tier is well-suited to a static/SPA frontend and does not sleep.
- The FAQ dataset (`faqs.json`) is bundled with the backend deploy — updating FAQs requires a redeploy, not a database migration.

11. Known Limitations

- No synonym or acronym expansion — TF-IDF matches on literal word overlap, so abbreviations (e.g. “NLP”) will not match a spelled-out FAQ question unless the words overlap after preprocessing.
- Single-turn matching only — the chatbot does not use conversation context; each message is matched independently against the FAQ set.
- Conversation history is scoped to `sessionStorage` — not shared across devices or after the browser tab closes.
- Voice input and Text-to-Speech quality depend on the browser and OS (Web Speech API); not all browsers support Speech-to-Text (notably Firefox has limited support).
- The FAQ dataset is a static JSON file — updating FAQs requires editing the file and redeploying the backend; there is no admin UI.

12. Possible Future Improvements

- Upgrade matching to Sentence-BERT embeddings for semantic (not just lexical) similarity, so “NLP” would correctly match “Natural Language Processing.”
- Add an admin interface for adding/editing FAQs without touching the JSON file directly.
- Move FAQ storage to a proper database (e.g. SQLite/PostgreSQL) for easier querying and scaling.
- Add automated tests (pytest) covering the API endpoints documented in Section 7.

- Add lightweight multi-turn context so follow-up questions like “explain that in more detail” can resolve against the previous answer.

13. User Interface

